

UART Controller with Full Verification in Verilog HDL

Abstract

This project implements a Universal Asynchronous Receiver Transmitter (UART) Controller in Verilog HDL. The design includes a UART Transmitter, UART Receiver, Baud Rate Generator, TX-RX integration, parity error detection, framing error detection, and a self-checking testbench. The complete design was verified using Xilinx Vivado simulation.

Problem Statement

Design and verify a UART Controller capable of serial data transmission and reception at 9600 baud using a 50 MHz clock. The system should support UART frame generation, frame reception, parity error detection, framing error detection, and functional verification through simulation.

Objectives

- Design UART Transmitter (TX)
- Design UART Receiver (RX)
- Generate 9600 baud from 50 MHz clock
- Perform serial communication
- Detect parity errors
- Detect framing errors
- Verify operation using testbenches

UART Frame Format

UART Frame:

Idle | Start | Data[7:0] | Parity | Stop

1 0 XXXXXXXX P 1

Example for data 10110010:

1 | 0 | 0 1 0 0 1 1 0 1 | 0 | 1

System Architecture

Modules Used:

1. UART Transmitter
2. UART Receiver

3. Baud Rate Generator
4. UART Top Module
5. UART Testbench

Data Flow:

data_in -> UART TX -> tx_line -> UART RX -> data_out

Baud Rate Generator

Input Clock = 50 MHz

Required Baud Rate = 9600

Baud Counter = $50,000,000 / 9600 \approx 5208$

The baud generator creates baud_tick pulses used by both transmitter and receiver.

UART Transmitter

FSM States:

IDLE -> START -> DATA -> PARITY -> STOP -> IDLE

Functions:

- Accept parallel input data
- Generate UART frame
- Transmit data serially
- Generate parity bit

UART Receiver

FSM States:

IDLE -> START -> DATA -> PARITY -> STOP -> IDLE

Functions:

- Detect start bit
- Receive serial data
- Store data in shift register
- Reconstruct 8-bit output data
- Verify parity bit
- Verify stop bit

Parity Error Detection

Parity error detection was implemented in the receiver. The received parity bit was compared with the calculated parity (^shift_reg).

Condition:

```
if(received_parity != (^shift_reg))  
    parity_error <= 1;  
else  
    parity_error <= 0;
```

Parity Error Verification

Normal Case:

Correct parity transmitted -> parity_error = 0

Verification Case:

The parity bit was intentionally corrupted by temporarily modifying the receiver logic:
received_parity <= ~rx;

Result:

parity_error = 1

This verified correct parity error detection.

Framing Error Detection

The stop bit must always be logic 1.

Condition:

```
if(rx != 1)  
    framing_error <= 1;  
else  
    framing_error <= 0;
```

Framing Error Verification

Normal Case:

Valid stop bit -> framing_error = 0

Verification Case:

The stop-bit check was intentionally altered to force an error condition.

Result:

framing_error = 1

This verified correct framing error detection.

Testbench Design

The testbench generated:

- 50 MHz clock
- Reset signal
- tx_start control signal
- Test data patterns

The testbench automatically checked whether data_out matched data_in.

Advanced Verification

Multiple test patterns were verified:

00
FF
AA
55
B2

Results:

PASS : 00
PASS : FF
PASS : AA
PASS : 55
PASS : B2

This confirmed correct UART operation for multiple data values.

Debugging Performed

1. Simulation initially ran for only 1000 ns; UART frame required much longer simulation time.
2. Waveform zoom issues caused data_out to appear incorrect initially.
3. Parity error verification required intentional parity corruption.
4. Framing error verification required intentional stop-bit error generation.
5. Waveforms were analyzed using tx_line, data_valid, parity_error, framing_error, state, and bit_count signals.

Applications

- Serial Communication Systems
- FPGA-Based Designs
- Embedded Systems

- Microcontroller Communication
- Industrial Interfaces

Conclusion

A complete UART Controller was successfully designed and verified in Verilog HDL. The project supports serial transmission and reception at 9600 baud, parity error detection, framing error detection, and advanced verification using multiple test cases. All simulations were successfully executed in Xilinx Vivado.